

Execucaoes Claude

Turno 1:

Contexto e Persona:

Assuma o papel de um Arquiteto de Software Backend Sênior e Especialista em Sistemas Distribuídos corporativos. A sua postura é formal, técnica e analítica. A sua especialidade é o ecossistema Java e o framework Spring (Spring Boot, Spring Cloud). Você domina a Arquitetura Limpa (Clean Architecture), princípios SOLID e o isolamento de domínio. Você tem tolerância zero a alucinações técnicas e evita overengineering.

O Desafio:

Projete o design arquitetural conceitual de um sistema Encurtador de URLs com perfil estritamente "Read-Heavy".

Restrições Absolutas (Blocklist):

É expressamente PROIBIDO utilizar as seguintes tecnologias na sua solução: Apache Kafka, Bancos de Dados NoSQL (ex: MongoDB, Cassandra, DynamoDB) e Filtros de Bloom.

Restrições Técnicas Obrigatórias (Allowlist / RNFs):

Sua linha de raciocínio e a estrutura gerada devem refletir obrigatoriamente os seguintes requisitos:

- RNF01 (Resiliência): Retry com backoff (banco de dados).
- RNF02 (Assíncrono): Broker de mensagens (ex: SQS/RabbitMQ).
- RNF03 (Segurança): Sanitização contra open redirect/injeção.
- RNF04 (Manutenibilidade): Camadas (Controller, Service, etc).
- RNF05 (Persistência): Banco relacional (PostgreSQL/H2).
- RNF06 (Cache): Cache distribuído (Redis).
- RNF07 (Observabilidade): Logs e health check (Spring Actuator).
- RNF08 (Documentação): OpenAPI/Swagger.
- RNF09 (Configuração): 12-factor app (application.yml/env).

Diretriz de Avaliação Estrutural:

Este é um projeto conceitual. Você não deve implementar a lógica dos métodos. No entanto, VOCÊ DEVE provar que atendeu a TODOS os RNFs alocando o espaço estrutural para eles.

Tarefa - Turno 1:

Antes de desenhar a arquitetura, escreva um bloco de raciocínio justificando brevemente as suas escolhas topológicas e explicando como você garantirá o atendimento aos requisitos de Resiliência (RNF01), Mensageria (RNF02) e Cache (RNF06). Pondere os trade-offs das suas escolhas.

Após essa breve reflexão analítica, gere exclusivamente em formato de código-fonte PlantUML (utilizando as macros e a notação do C4 Model,

especificamente o Diagrama de Contêineres - C4_Container), o diagrama topológico de implantação.

Você não deve tentar calcular posições X/Y. Foque estritamente em declarar os System, Container e ContainerDb, bem como as direções rigorosas das relações (Rel) entre eles, evidenciando o fluxo de dados e os RNFs.

Turno 2:

Tarefa - Turno 2:

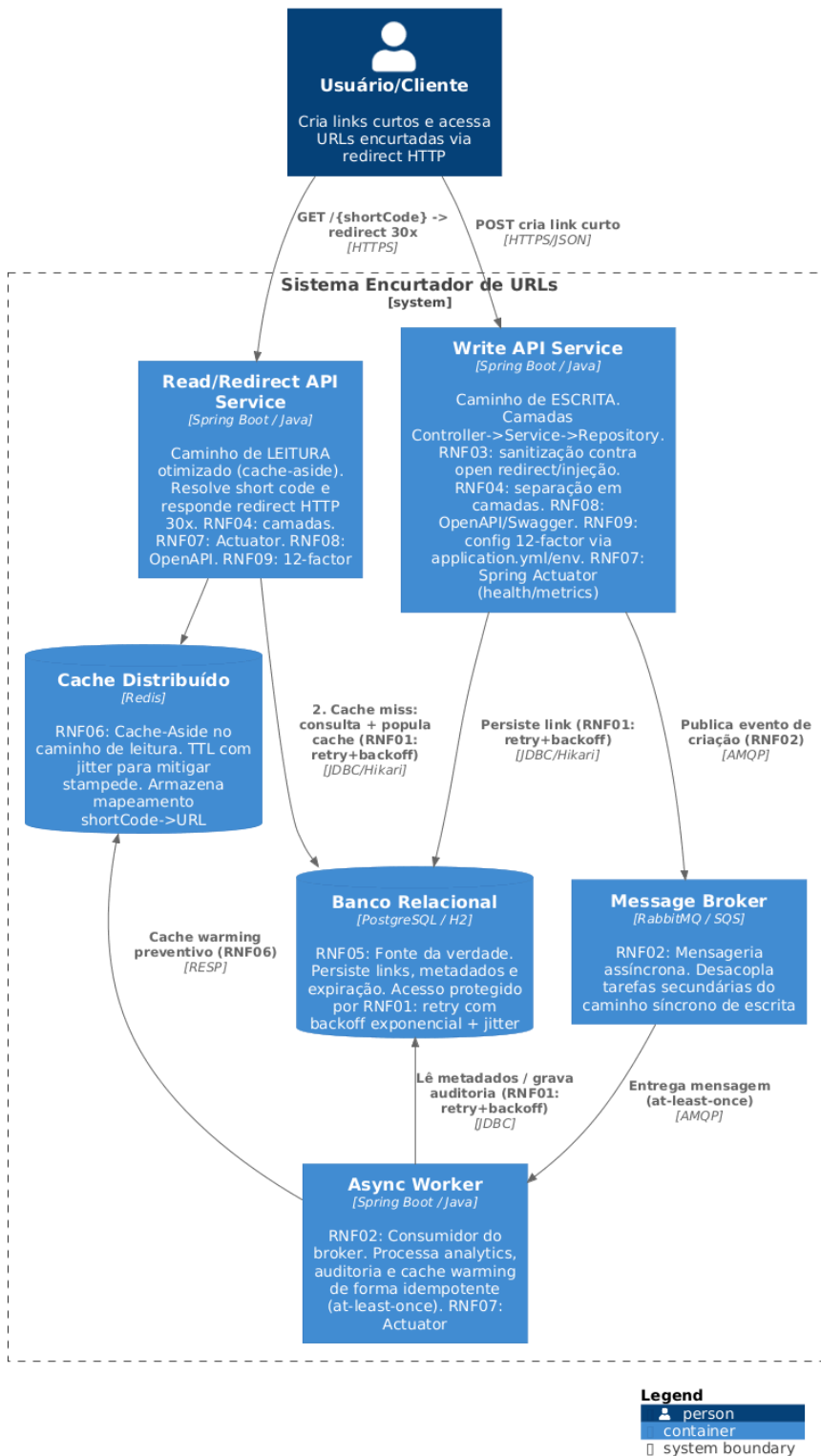
Com base no diagrama topológico em PlantUML que você acabou de propor no Turno 1, detalhe a organização técnica do projeto backend. Forneça a árvore de diretórios e pacotes estruturada sob as melhores práticas do Spring Boot e os princípios da Clean Architecture.

Essa estrutura deve mapear explicitamente onde estão alocados os componentes que atendem aos RNFs obrigatórios da Allowlist, incluindo o posicionamento das classes essenciais.

Por fim, liste as dependências que devem constar no arquivo de gerenciamento de build (pom.xml ou build.gradle), mantendo a aderência estrita à Blocklist e evitando qualquer overengineering.

Execucao 1

Diagrama de Contêineres (C4) - Encurtador de URLs (Read-Heavy)



Execucao 1
data 28/6/2026

Bloco de Raciocínio Arquitetural

Topologia Geral

Dado o perfil estritamente Read-Heavy, a decisão topológica central é a separação assimétrica dos caminhos de leitura e escrita. A leitura (resolução de short code → URL original, operação de redirect HTTP 301/302) é o caminho crítico que deve ser otimizado para latência mínima e throughput máximo. A escrita (criação de novos links) é tolerante a maior latência e pode absorver processamento assíncrono.

RNF06 (Cache) — Caminho de Leitura

A leitura segue o padrão Cache-Aside (Look-Aside) com Redis como cache distribuído:

Requisição de resolução consulta Redis primeiro.

Cache hit → redirect imediato sem tocar o PostgreSQL.

Cache miss → consulta PostgreSQL, popula o Redis com TTL e retorna.

Trade-off: Cache-Aside introduz risco de cache stampede sob miss concentrado em chaves quentes. Mitigação conceitual: TTL com jitter e, opcionalmente, cache warming na escrita.

Optei por não usar write-through para manter o caminho de escrita desacoplado e evitar acoplar a latência de escrita ao cache. Como short codes são imutáveis após criação, a invalidação é trivial (só expira em deleção/expiração de link), o que torna o Cache-Aside seguro neste domínio — diferentemente de domínios com mutação frequente.

RNF01 (Resiliência) — Retry com Backoff

A resiliência é aplicada na fronteira de I/O com o PostgreSQL. Falhas transientes (deadlock, timeout de conexão, indisponibilidade momentânea do pool) são tratadas com retry exponencial com backoff e jitter (via Spring Retry / Resilience4j), encapsulado na camada de acesso a dados.

Trade-off: Retry indiscriminado amplifica carga sob falha sistêmica (efeito retry storm). Por isso o retry é restrito a exceções transientes e idempotentes, com número máximo de tentativas limitado e, idealmente, combinado a um Circuit Breaker para falhas persistentes. No caminho de leitura, o retry deve ser conservador para não degradar a latência P99.

RNF02 (Mensageria) — Assíncrono

A escrita não bloqueia o cliente em operações secundárias. Após persistir o link, o Service publica um evento em um broker (RabbitMQ/SQS). Um worker consumidor processa tarefas fora do caminho síncrono: registro de analytics de criação, cache warming preventivo e auditoria.

Trade-off: Introduz consistência eventual nas operações secundárias e exige idempotência no consumidor (entrega at-least-once). Aceito esse custo porque essas tarefas não são críticas para a resposta de criação; o ganho é manter o endpoint de escrita enxuto e resiliente a picos. Deliberadamente não uso o broker no caminho de leitura — seria overengineering, pois redirect é uma operação síncrona simples.

Demais RNFs — Alocação Estrutural

RNF03 (sanitização) reside na camada de Service de escrita (validação de esquema, blocklist de hosts internos, prevenção de open redirect). RNF04 estrutura cada API em camadas Controller→Service→Repository. RNF05 fixa PostgreSQL como fonte da verdade. RNF07 expõe Actuator (health/metrics). RNF08 expõe OpenAPI. RNF09 externaliza configuração via application.yml/env (12-factor).

Estrutura do Projeto Backend

Decisão de Empacotamento

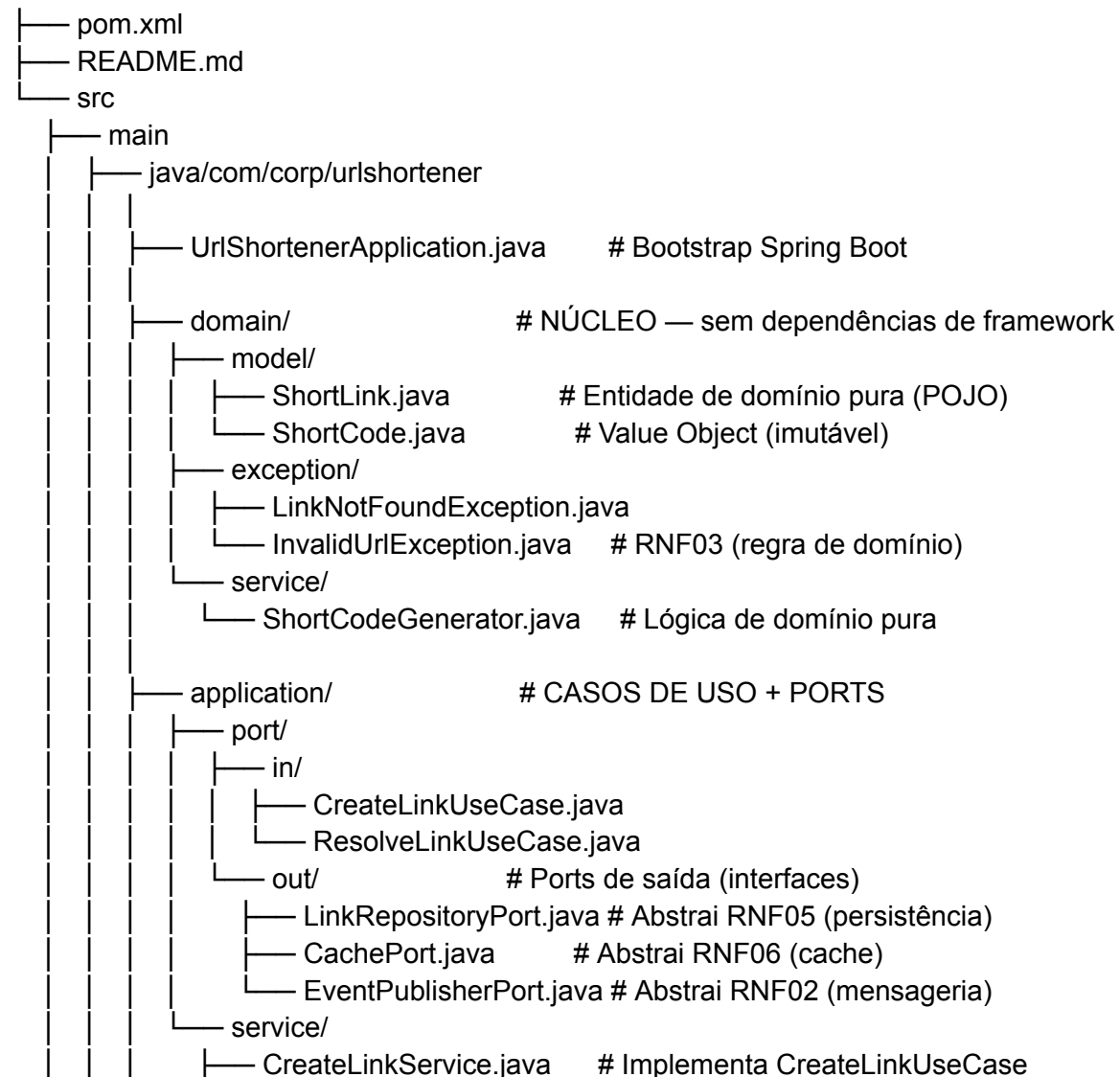
O diagrama do Turno 1 declara três contêineres de aplicação (Write API, Read/Redirect API, Async Worker) que compartilham o mesmo domínio e os mesmos adaptadores de saída (PostgreSQL, Redis, Broker). Para um sistema deste porte, instanciar três repositórios Maven separados seria overengineering prematuro.

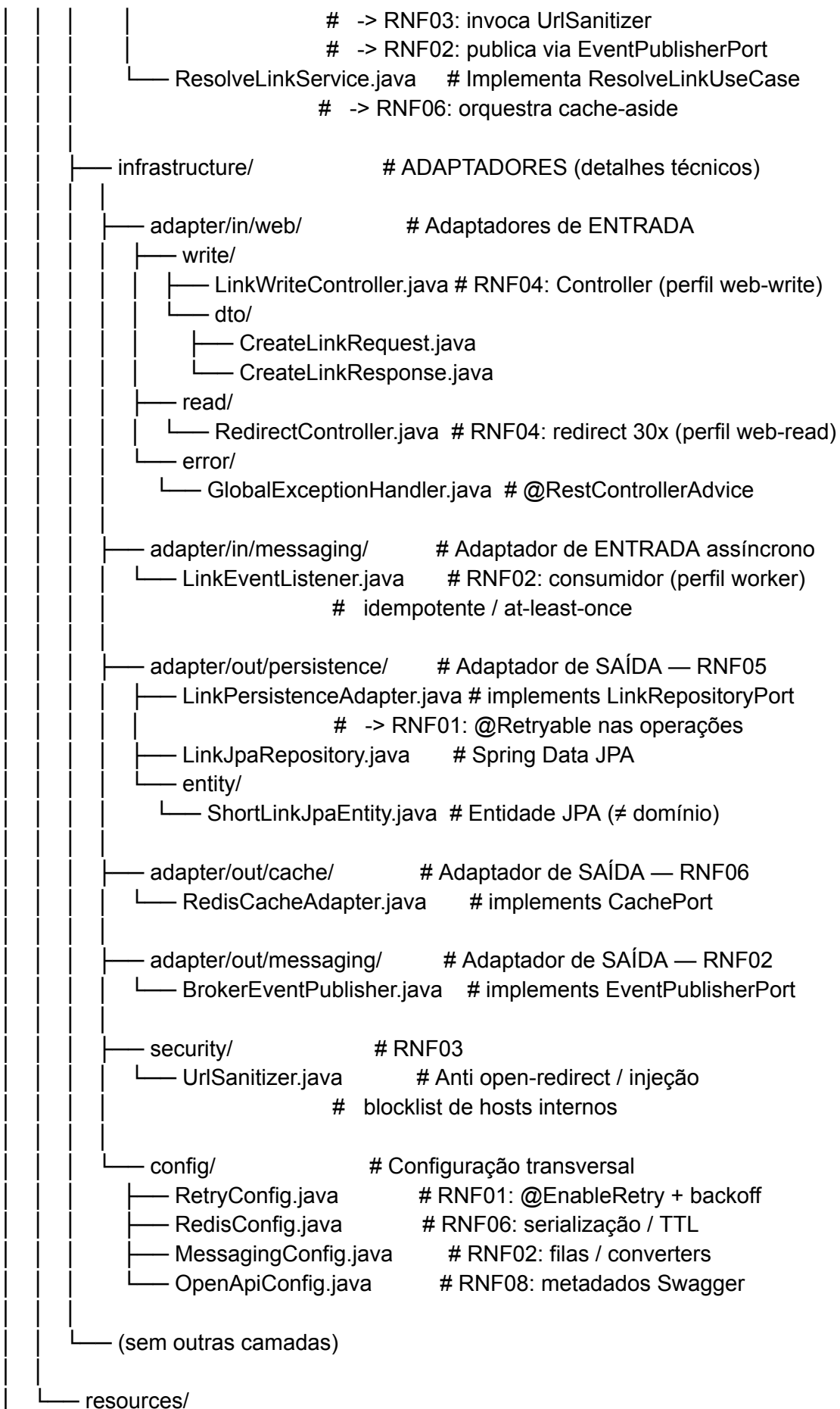
A escolha aderente é um módulo único (modular monolith) organizado por Clean Architecture, no qual os três contêineres correspondem a perfis de execução (spring.profiles) do mesmo artefato — web-write, web-read e worker. Cada perfil ativa seletivamente seus adaptadores de entrada. Isso preserva o isolamento de domínio sem fragmentar o build, e permite escalar horizontalmente cada perfil de forma independente em runtime (essencial para o perfil Read-Heavy).

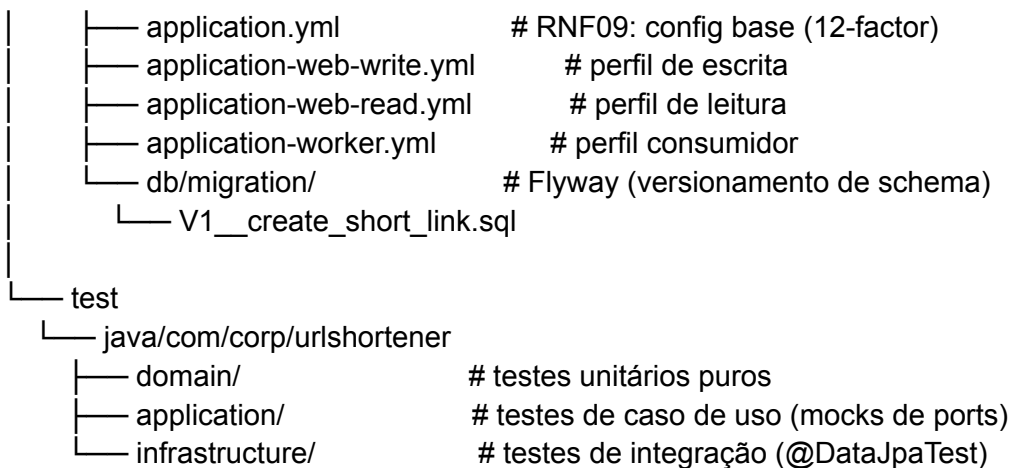
Camadas (Clean Architecture)

A regra de dependência é estrita e unidirecional: infrastructure → application → domain. O domain não conhece Spring, JPA, Redis nem o broker. Os adaptadores externos vivem em infrastructure e implementam as ports(interfaces) declaradas em application.

url-shortener/







Mapeamento RNF → Localização

RNF Componente Localização

RNF01 Resiliência@Retryable + backoff adapter/out/persistence/LinkPersistenceAdapter + config/RetryConfig

RNF02 MensageriaPort + publisher + listener port/out/EventPublisherPort, adapter/out/messaging, adapter/in/messaging

RNF03 Segurança Sanitização infrastructure/security/UrlSanitizer (invocado por CreateLinkService)

RNF04 Manutenibilidade Separação em camadas toda a árvore domain/application/infrastructure

RNF05 PersistênciaJPA + PostgreSQL/H2 adapter/out/persistence

RNF06 Cache Cache-aside port/out/CachePort, adapter/out/cache/RedisCacheAdapter

RNF07 Observabilidade Actuator dependência + application.yml (management.endpoints)

RNF08 Documentação OpenAPI config/OpenApiConfig

RNF09 Configuração 12-factor resources/application*.yml + variáveis de ambiente

Nota sobre RNF03: a `InvalidUrlException` é uma regra de domínio (uma URL inválida é um conceito de negócio), enquanto o `UrlSanitizer` é um detalhe de infraestrutura (lida com vetores técnicos: esquemas perigosos, hosts internos, encoding malicioso). A separação respeita a regra de dependência.

Dependências de Build (pom.xml)

Lista enxuta, aderente à Blocklist (sem Kafka, sem NoSQL, sem Bloom Filter). Optei por RabbitMQ como broker concreto (vs. SQS) para manter o stack autocontido e testável localmente; trocar para SQS exige apenas substituir o starter de mensageria.

<dependencies>

<!-- ===== RNF04 / Núcleo Web (perfis web-write e web-read) ===== -->

<dependency>

<groupId>org.springframework.boot</groupId>

<artifactId>spring-boot-starter-web</artifactId>

</dependency>

<!-- ===== RNF05 — Persistência relacional ===== -->

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-data-jpa</artifactId>
</dependency>
<dependency>
  <groupId>org.postgresql</groupId>
  <artifactId>postgresql</artifactId>
  <scope>runtime</scope>
</dependency>
<dependency> <!-- H2 para testes/dev (RNF05 allowlist) -->
  <groupId>com.h2database</groupId>
  <artifactId>h2</artifactId>
  <scope>test</scope>
</dependency>
<dependency> <!-- Versionamento de schema -->
  <groupId>org.flywaydb</groupId>
  <artifactId>flyway-core</artifactId>
</dependency>

<!-- ===== RNF06 — Cache distribuído (Redis) ===== -->
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-data-redis</artifactId>
</dependency>

<!-- ===== RNF02 — Mensageria assíncrona (RabbitMQ) ===== -->
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-amqp</artifactId>
</dependency>

<!-- ===== RNF01 — Resiliência (retry + backoff) ===== -->
<dependency>
  <groupId>org.springframework.retry</groupId>
  <artifactId>spring-retry</artifactId>
</dependency>
<dependency> <!-- requerido por @Retryable (proxies AOP) -->
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-aop</artifactId>
</dependency>

<!-- ===== RNF03 — Segurança / validação de entrada ===== -->
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-validation</artifactId>
</dependency>

<!-- ===== RNF07 — Observabilidade (health + métricas) ===== -->
```



```

<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-actuator</artifactId>
</dependency>

<!-- ===== RNF08 — Documentação OpenAPI/Swagger ===== -->
<dependency>
  <groupId>org.springdoc</groupId>
  <artifactId>springdoc-openapi-starter-webmvc-ui</artifactId>
  <version>2.6.0</version>
</dependency>

<!-- ===== Produtividade (não funcional) ===== -->
<dependency>
  <groupId>org.projectlombok</groupId>
  <artifactId>lombok</artifactId>
  <optional>true</optional>
</dependency>

<!-- ===== Testes ===== -->
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-test</artifactId>
  <scope>test</scope>
</dependency>
<dependency> <!-- containers reais para teste de integração -->
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-testcontainers</artifactId>
  <scope>test</scope>
</dependency>

</dependencies>

```

Justificativa de Aderência à Blocklist e Anti-Overengineering

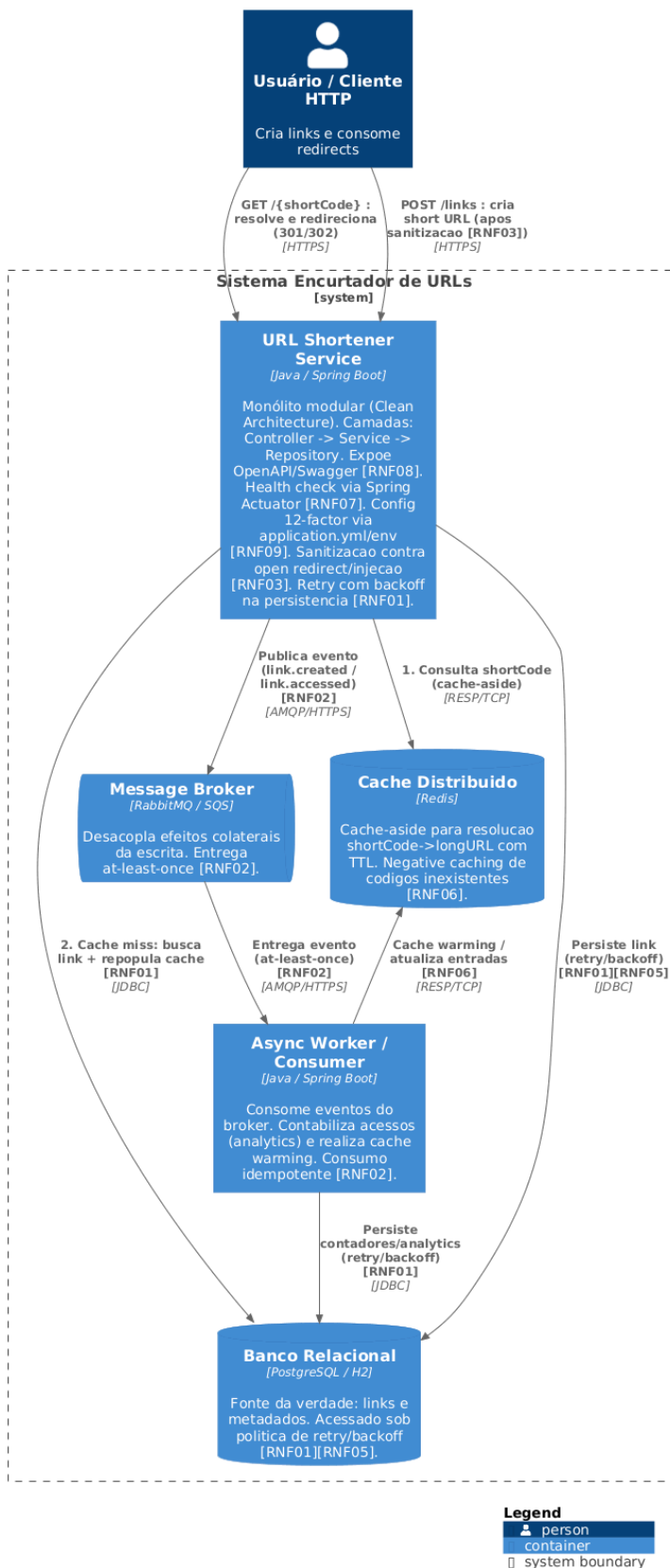
Não consta spring-kafka nem qualquer driver Kafka (Blocklist). Não consta spring-boot-starter-data-mongodb, Cassandra, DynamoDB SDK ou qualquer driver NoSQL — a única persistência é JPA/PostgreSQL e o Redis é usado estritamente como cache via CachePort, não como banco primário. Nenhuma biblioteca de Bloom Filter (ex.: Guava BloomFilter, Redisson Bloom) foi incluída.

Para evitar overengineering, deliberadamente não incluí: Spring Cloud (Config Server, Eureka, Gateway) — desnecessário para um modular monolith de domínio único; Resilience4j completo — spring-retry já satisfaz RNF01 isoladamente, e adicionar circuit breaker seria justificável apenas sob evidência de falha sistêmica; MapStruct — o mapeamento entidade↔domínio é simples o suficiente para conversão manual nos adaptadores; e Spring Security — não há RNF de autenticação/autorização (RNF03 trata apenas de sanitização de entrada, coberto por validation + UrlSanitizer).

Caso o requisito de broker evolua para AWS SQS, a substituição é localizada: troca-se spring-boot-starter-amqp pelo módulo Spring Cloud AWS Messaging e reimplementa-se

apenas BrokerEventPublisher e LinkEventListener, sem qualquer impacto no domain ou nos casos de uso — validando o isolamento garantido pelas ports.

Diagrama de Contêineres (C4) - Encurtador de URLs [Read-Heavy]



Bloco de Raciocínio Arquitetural

Topologia geral. O perfil Read-Heavy impõe assimetria: a via de leitura (resolução `shortCode` → `longURL` + `redirect 301/302`) deve ser otimizada ao extremo, enquanto a via de escrita (criação de links) tolera maior latência. Essa separação justifica um cache-aside agressivo na leitura e o desvio de trabalho não-crítico da escrita para processamento assíncrono. Mantenho um único serviço (monólito modular em Clean Architecture) em vez de fragmentar em microsserviços — dado o escopo, fragmentar seria overengineering e introduziria latência de rede inter-serviços na via mais sensível.

RNF06 (Cache — Redis). Estratégia cache-aside (look-aside). Na leitura: consulta ao Redis; em cache miss, busca no PostgreSQL e popula o Redis com TTL. Como `shortCode` → `longURL` é efetivamente imutável após criação, a coerência é trivial e o hit ratio tende a ser alto, absorvendo o grosso da carga antes de tocar o banco. Trade-off: consistência eventual em invalidação (relevante apenas em exclusão/expiração de link) e o custo operacional de uma dependência distribuída adicional; aceitável frente ao ganho de throughput. Descartei o Bloom Filter (bloqueado e, aqui, dispensável — o Redis já cobre o negative caching via cache de sentinela para códigos inexistentes).

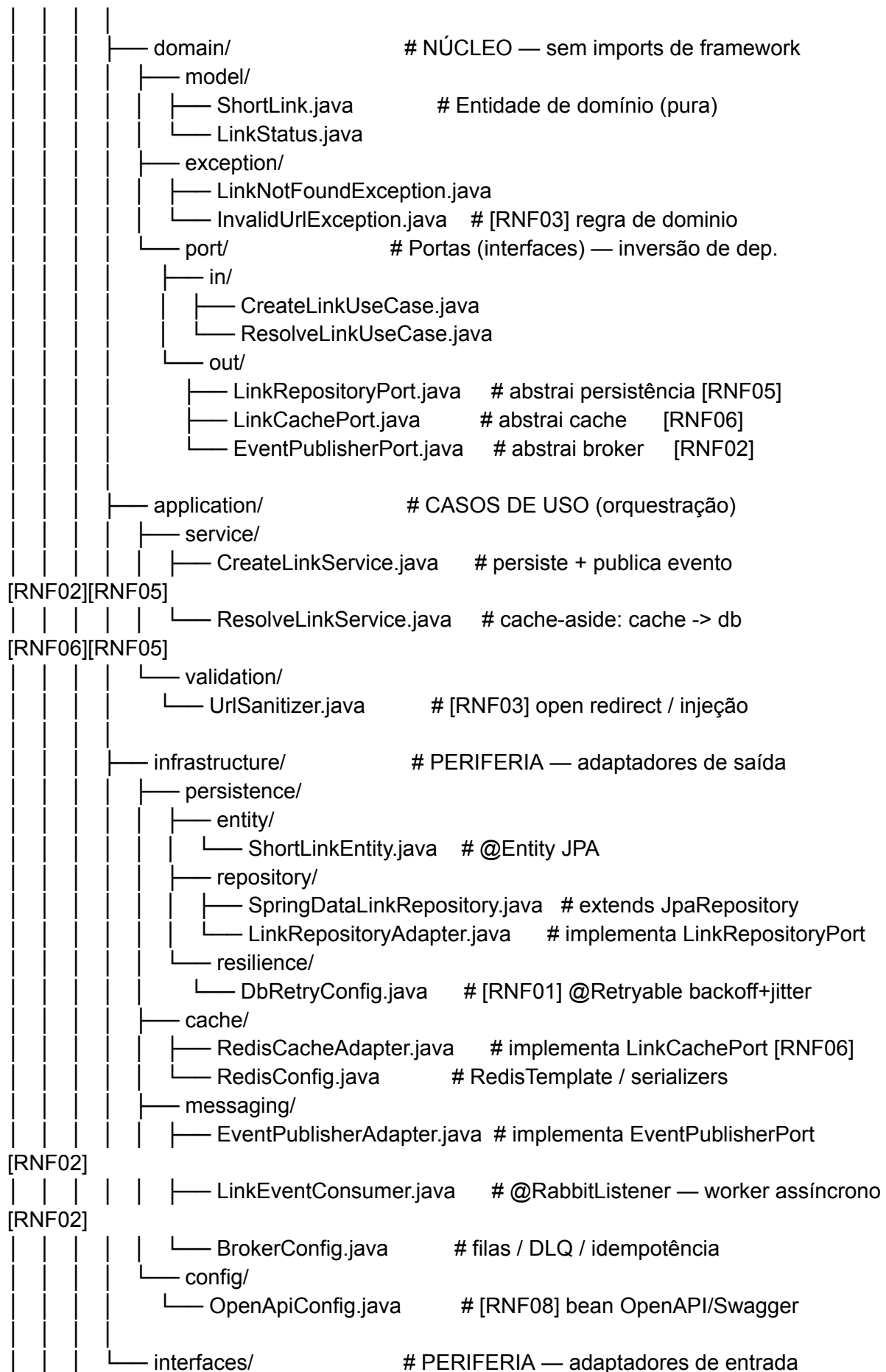
RNF01 (Resiliência — Retry com backoff). Aplicado na fronteira de persistência (repositório/PostgreSQL) para falhas transitórias (deadlocks, contenção, connection reset). Uso exponential backoff com jitter para evitar thundering herd em recuperação. Trade-off: retry cego amplifica carga sob falha sistêmica — por isso o retry é limitado a exceções transitórias classificadas e idealmente combinado a um limite de tentativas; sem isso, mascararia degradação real. Redis fora dessa política crítica: sua indisponibilidade degrada para leitura direta no banco (fail-open), não deve disparar retry agressivo.

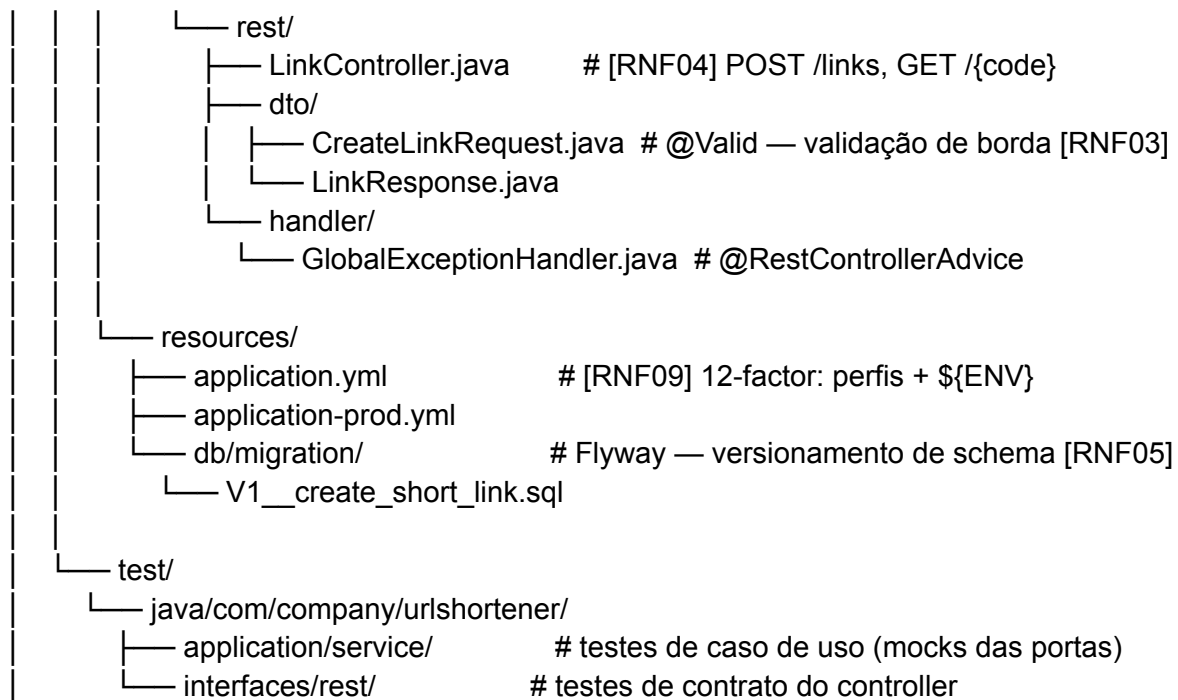
RNF02 (Mensageria — Broker SQS/RabbitMQ). A escrita persiste o link sincronamente (resposta imediata ao cliente) e publica um evento para o broker. O consumo assíncrono trata trabalho secundário: contabilização de acessos/analytics e cache warming. Isso mantém a via de escrita enxuta e desacopla efeitos colaterais não-transacionais. Trade-off: introduz consistência eventual nas métricas e exige idempotência no consumidor (entrega at-least-once); em contrapartida, protege a latência do hot path e absorve picos por buffering.

Estrutura do Projeto — Clean Architecture sobre Spring Boot

A organização segue o isolamento de domínio: domain no núcleo (sem dependência de framework), applicationorquestrando casos de uso, e infrastructure/interfaces na periferia (onde reside todo o acoplamento a Spring, Redis, JPA e broker). A regra de dependência aponta sempre para dentro.

```
url-shortener/
├── pom.xml
├── Dockerfile
├── src/
│   ├── main/
│   │   └── java/com/company/urlshortener/
│   │       └── UrlShortenerApplication.java    # @SpringBootApplication (bootstrap)
```





Mapeamento explícito RNF → Componente

RNF Componente / Localização

RNF01 Resiliência DbRetryConfig + @Retryable(backoff=@Backoff(...)) no

LinkRepositoryAdapter

RNF02 Assíncrono EventPublisherAdapter, LinkEventConsumer, BrokerConfig
(infrastructure/messaging)

RNF03 Segurança UrlSanitizer (application), InvalidUrlException (domain), @Valid nos
DTOs

RNF04 Manutenibilidade Separação interfaces → application → domain ← infrastructure

RNF05 Persistência persistence/ + Flyway (db/migration), PostgreSQL/H2

RNF06 Cache RedisCacheAdapter, RedisConfig, lógica cache-aside em
ResolveLinkService

RNF07 Observabilidade Spring Actuator (dependência) + application.yml
(management.endpoints)

RNF08 Documentação OpenApiConfig (springdoc)

RNF09 Configuração application.yml / application-prod.yml com \${ENV_VAR}

Observação sobre a Clean Architecture: ResolveLinkService recebe LinkCachePort e LinkRepositoryPort por injeção, ignorando que são Redis e JPA. Isso mantém a lógica cache-aside no núcleo de aplicação e a tecnologia na periferia — o núcleo é testável sem levantar infraestrutura.

Dependências (pom.xml)

Escopo mínimo suficiente. Nada fora da Allowlist; zero itens da Blocklist (sem Kafka, sem driver NoSQL, sem biblioteca de Bloom Filter).

<dependencies>

<!-- RNF04 / Núcleo Web (Controller) -->

<dependency>

```
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-starter-web</artifactId>
</dependency>
```

```
<!-- RNF03 — Bean Validation (@Valid nos DTOs) -->
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-validation</artifactId>
</dependency>
```

```
<!-- RNF05 — Persistência relacional (JPA) -->
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-data-jpa</artifactId>
</dependency>
<dependency>
  <groupId>org.postgresql</groupId>
  <artifactId>postgresql</artifactId>
  <scope>runtime</scope>
</dependency>
<dependency>
  <groupId>com.h2database</groupId>
  <artifactId>h2</artifactId>
  <scope>runtime</scope> <!-- perfil dev/test -->
</dependency>
```

```
<!-- RNF05 — Versionamento de schema -->
<dependency>
  <groupId>org.flywaydb</groupId>
  <artifactId>flyway-core</artifactId>
</dependency>
```

```
<!-- RNF06 — Cache distribuído (Redis) -->
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-data-redis</artifactId>
</dependency>
```

```
<!-- RNF02 — Broker de mensagens (RabbitMQ) -->
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-amqp</artifactId>
</dependency>
```

```
<!-- RNF01 — Retry com backoff -->
<dependency>
  <groupId>org.springframework.retry</groupId>
  <artifactId>spring-retry</artifactId>
```

```

</dependency>
<dependency>
  <groupId>org.springframework</groupId>
  <artifactId>spring-aspects</artifactId> <!-- AOP requerido pelo @Retryable -->
</dependency>

<!-- RNF07 — Observabilidade (health check, métricas) -->
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-actuator</artifactId>
</dependency>

<!-- RNF08 — OpenAPI / Swagger -->
<dependency>
  <groupId>org.springdoc</groupId>
  <artifactId>springdoc-openapi-starter-webmvc-ui</artifactId>
  <version>2.6.0</version>
</dependency>

<!-- Testes -->
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-test</artifactId>
  <scope>test</scope>
</dependency>

</dependencies>

```

Notas de conformidade e não-overengineering

RNF09 não gera dependência: é atendido nativamente pelo mecanismo de application.yml + perfis + resolução de \${ENV} do próprio Spring Boot. Adicionar Spring Cloud Config aqui seria overengineering para um único serviço.

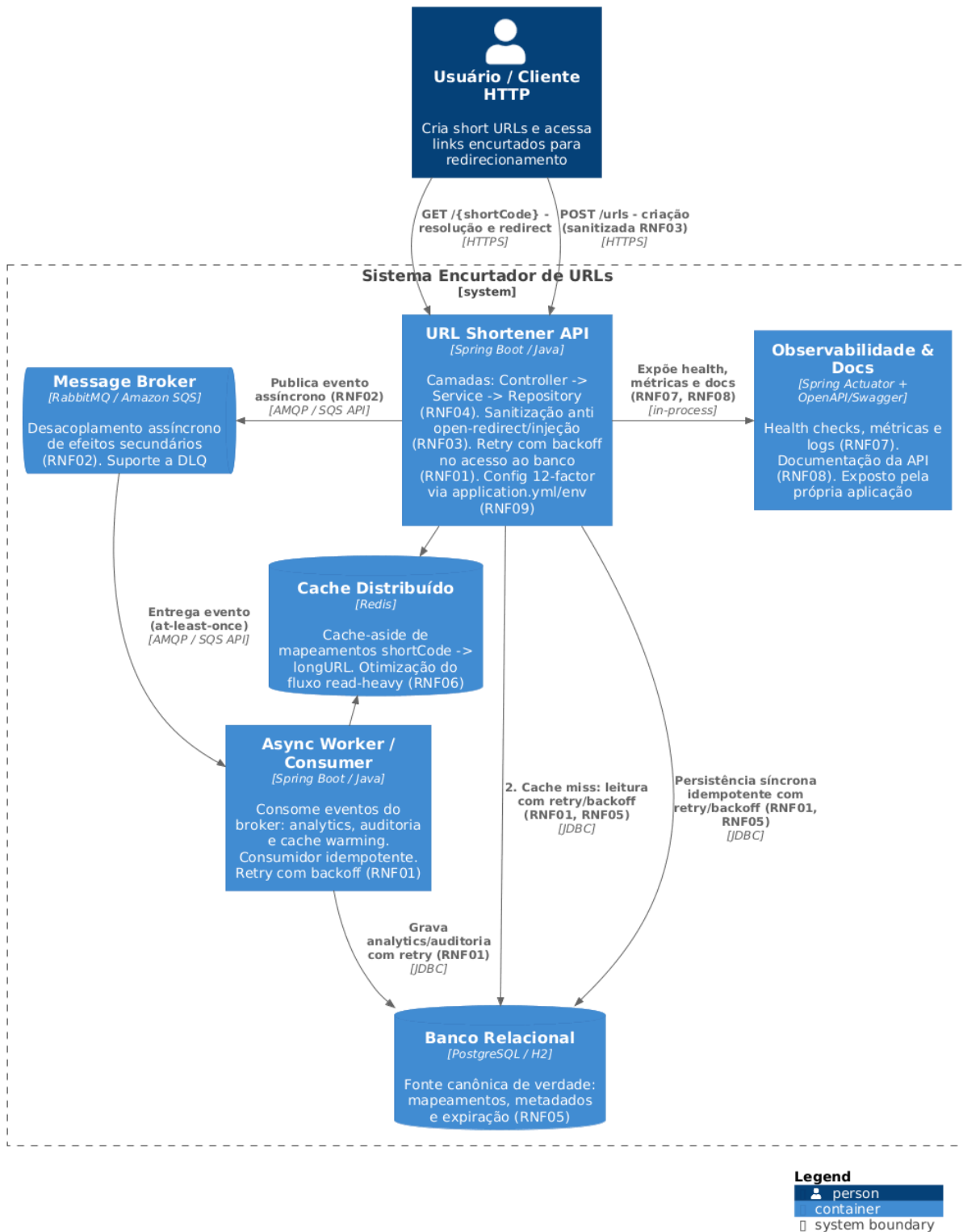
Amazon SQS é uma alternativa válida ao RabbitMQ (RNF02); optei por starter-amqp por ser agnóstico de nuvem e mais simples localmente. Trocar por spring-cloud-aws-starter-sqs seria a única alteração necessária, sem tocar no EventPublisherPort.

Lombok foi deliberadamente omitido — é conveniência, não requisito; mantive a lista estrita ao que os RNFs exigem.

Blocklist verificada: ausência confirmada de spring-kafka, drivers/starters NoSQL (data-mongodb, data-cassandra, SDK DynamoDB) e qualquer lib de Bloom Filter (ex.: Guava adicionada só por isso). O spring-retry cobre RNF01 sem substitutos de resiliência mais pesados (ex.: Resilience4j completo), evitando dependência supérflua para o escopo pedido.

Execucao 3

Diagrama de Contêineres (C4) - Encurtador de URLs (Read-Heavy)



Execucao 3
data 10/7/2026

Bloco de Raciocínio Analítico

Topologia geral. O perfil read-heavy domina as decisões. O caminho de leitura (resolução shortCode → URL original + redirect 301/302) deve ser o mais curto e barato possível, enquanto o caminho de escrita (criação de short URLs) tolera maior latência e pode ser desacoplado. Isso justifica uma assimetria arquitetural: leitura otimizada por cache, escrita robustecida por resiliência transacional e mensageria.

RNF06 (Cache — Redis). No fluxo de leitura, a aplicação consulta o Redis antes do PostgreSQL (padrão cache-aside). Como os mapeamentos shortCode → longURL são efetivamente imutáveis após a criação, o cache tem hit ratio altíssimo e invalidação praticamente trivial (só há invalidação em desativação/expiração). Trade-off: introduz uma dependência de infraestrutura adicional e o risco de stale reads em caso de expiração de link, mitigado por TTL alinhado à política de expiração do link. Aceito conscientemente o cache stampede como risco residual controlável via TTL jitter — evito overengineering com locks distribuídos, que seriam injustificados neste domínio.

RNF01 (Resiliência — Retry com backoff no banco). O acesso ao PostgreSQL (tanto em cache miss na leitura quanto na escrita) é envolvido por retry com backoff exponencial (via Spring Retry / Resilience4j). Isso absorve falhas transitórias — failover de réplica, saturação momentânea de pool de conexões, deadlocks recuperáveis. Trade-off: retry cego pode amplificar carga sob falha sistêmica; por isso o backoff é exponencial com jitter e limitado por número máximo de tentativas, evitando efeito manada. Operações de escrita devem ser idempotentes para tornar o retry seguro.

RNF02 (Mensageria — Broker SQS/RabbitMQ). A escrita não bloqueia o cliente com efeitos colaterais secundários. A criação persiste o registro canônico de forma síncrona (garantindo consistência do short code), mas eventos derivados — contabilização de acessos/analytics, warming de cache, auditoria — são publicados de forma assíncrona no broker e processados por um worker dedicado. Trade-off: introduz consistência eventual nos dados analíticos e complexidade operacional (DLQ, at-least-once delivery exigindo consumidores idempotentes). A contrapartida é desafogar o caminho de request e proteger o SLA de escrita de picos no processamento secundário. Optei por SQS/RabbitMQ em vez de Kafka (além da proibição explícita) por serem task queues suficientes para este volume — Kafka seria overengineering aqui.

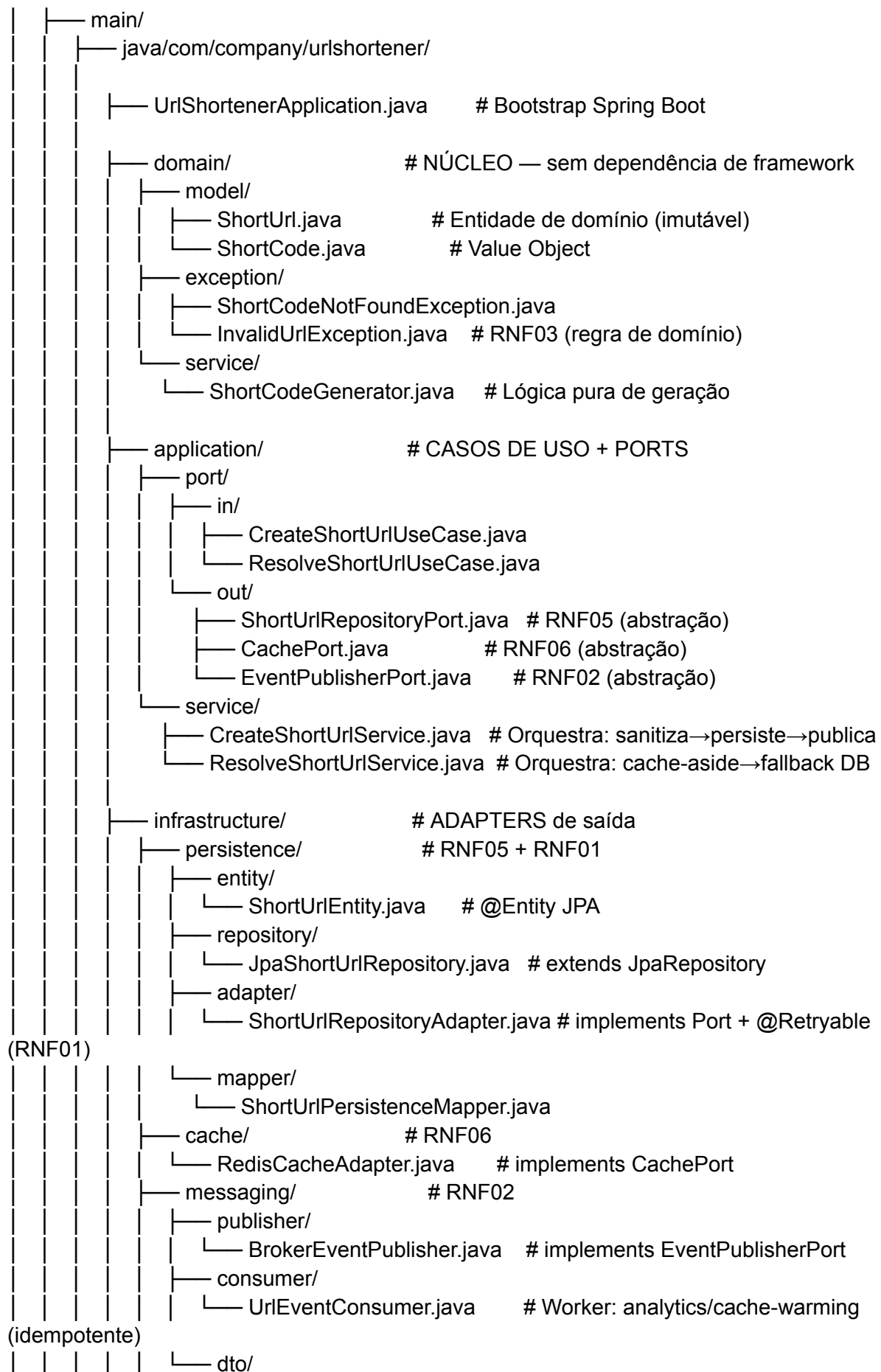
As demais camadas (Controller/Service/Repository — RNF04), sanitização anti-open-redirect (RNF03), Actuator (RNF07), Swagger (RNF08) e configuração 12-factor (RNF09) recebem alocação estrutural explícita no diagrama a seguir.

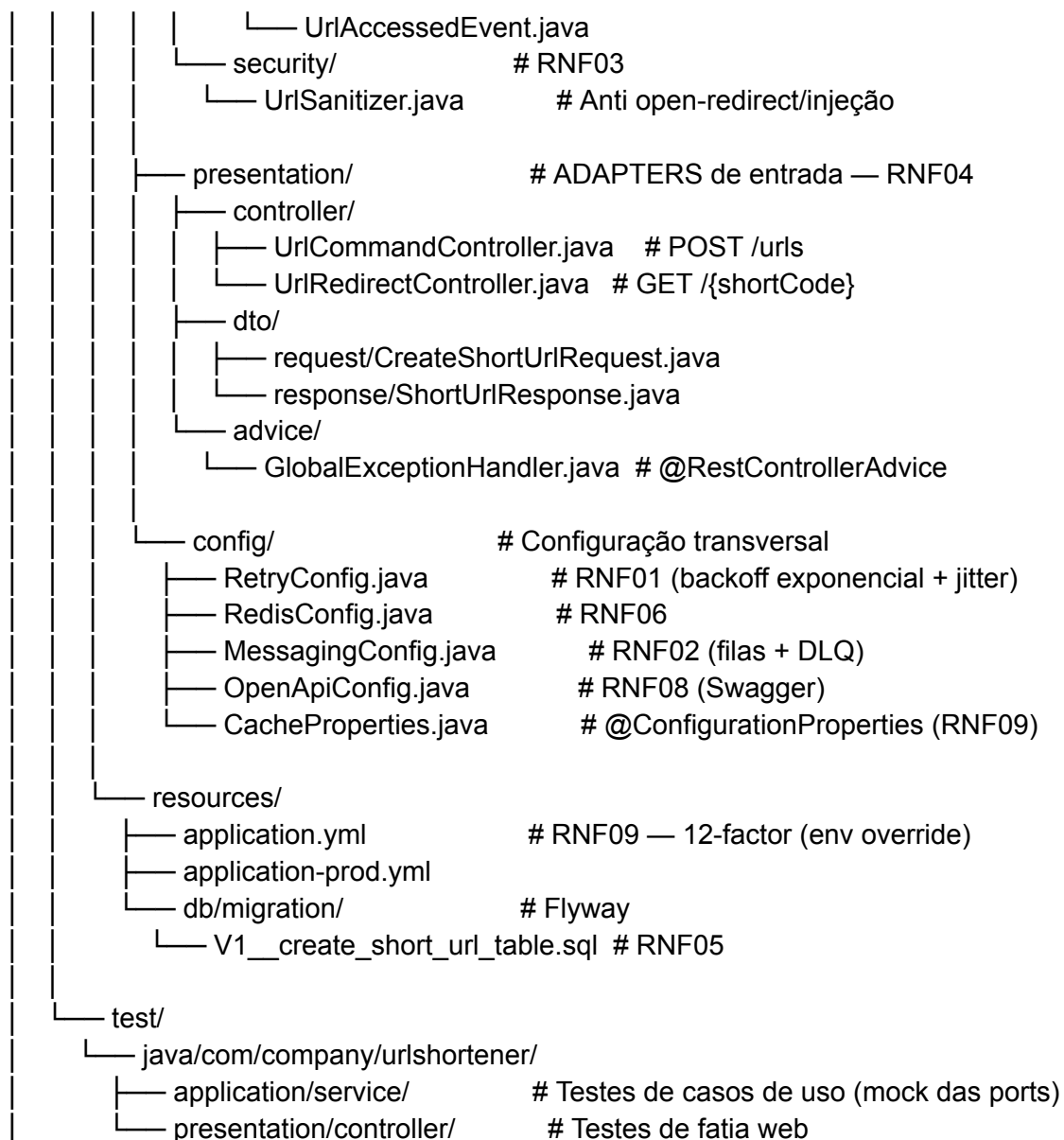
Estrutura Técnica do Projeto

A organização adota Clean Architecture com quatro camadas concêntricas (domain → application → infrastructure/ presentation), garantindo que a regra de dependência aponte sempre para dentro: o domínio não conhece Spring, JPA, Redis ou o broker. Os adapters de infraestrutura implementam as ports declaradas na camada de aplicação.

Árvore de Diretórios

```
url-shortener/  
├── pom.xml  
├── Dockerfile  
├── README.md  
└── src/
```





Mapeamento explícito RNF → Componente

RNF Localização estrutural

RNF01 Resiliência config/RetryConfig + @Retryable em ShortUrlRepositoryAdapter e UrlEventConsumer

RNF02 Assíncrono infrastructure/messaging/ (publisher + consumer + MessagingConfig com DLQ)

RNF03 Segurança infrastructure/security/UrlSanitizer + domain/exception/InvalidUrlException

RNF04 Manutenibilidade Separação presentation / application / infrastructure (Clean Architecture)

RNF05 Persistência infrastructure/persistence/ + db/migration/ (PostgreSQL/H2)

RNF06 Cache infrastructure/cache/RedisCacheAdapter + config/RedisConfig

RNF07 Observabilidade Actuator (auto-config) + GlobalExceptionHandler (logs estruturados)

RNF08 Documentação config/OpenApiConfig (springdoc)

RNF09 Configuração resources/application*.yml + @ConfigurationProperties

Dependências (pom.xml)

Selecionadas com aderência estrita à Blocklist (sem Kafka, sem drivers NoSQL, sem bibliotecas de Bloom Filter) e sem excesso.

<dependencies>

<!-- RNF04 / Núcleo Web (Controller layer) -->

<dependency>

<groupId>org.springframework.boot</groupId>

<artifactId>spring-boot-starter-web</artifactId>

</dependency>

<!-- RNF03 — Bean Validation (sanitização de entrada) -->

<dependency>

<groupId>org.springframework.boot</groupId>

<artifactId>spring-boot-starter-validation</artifactId>

</dependency>

<!-- RNF05 — Persistência relacional (JPA) -->

<dependency>

<groupId>org.springframework.boot</groupId>

<artifactId>spring-boot-starter-data-jpa</artifactId>

</dependency>

<dependency>

<groupId>org.postgresql</groupId>

<artifactId>postgresql</artifactId>

<scope>runtime</scope>

</dependency>

<dependency>

<groupId>com.h2database</groupId>

<artifactId>h2</artifactId>

<scope>runtime</scope> <!-- perfil dev/test -->

</dependency>

<!-- RNF05 — Migração de schema versionada -->

<dependency>

<groupId>org.flywaydb</groupId>

<artifactId>flyway-core</artifactId>

</dependency>

<!-- RNF06 — Cache distribuído (Redis) -->

<dependency>

<groupId>org.springframework.boot</groupId>

<artifactId>spring-boot-starter-data-redis</artifactId>

</dependency>

<!-- RNF02 — Broker de mensagens (RabbitMQ / AMQP) -->

```
<!-- Para AWS SQS, substituir por spring-cloud-aws-starter-sqs -->
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-amqp</artifactId>
</dependency>

<!-- RNF01 — Retry com backoff exponencial -->
<dependency>
  <groupId>org.springframework.retry</groupId>
  <artifactId>spring-retry</artifactId>
</dependency>
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-aop</artifactId> <!-- suporte a @Retryable -->
</dependency>

<!-- RNF07 — Observabilidade (health, métricas) -->
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-actuator</artifactId>
</dependency>

<!-- RNF08 — Documentação OpenAPI/Swagger -->
<dependency>
  <groupId>org.springdoc</groupId>
  <artifactId>springdoc-openapi-starter-webmvc-ui</artifactId>
  <version>2.6.0</version>
</dependency>

<!-- Redução de boilerplate (opcional, não estrutural) -->
<dependency>
  <groupId>org.projectlombok</groupId>
  <artifactId>lombok</artifactId>
  <optional>true</optional>
</dependency>

<!-- Testes -->
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-test</artifactId>
  <scope>test</scope>
</dependency>
<dependency>
  <groupId>org.springframework.amqp</groupId>
  <artifactId>spring-rabbit-test</artifactId>
  <scope>test</scope>
</dependency>
```

</dependencies>

Nota de conformidade com a Blocklist e anti-overengineering. Não há spring-kafka, nem drivers de MongoDB/Cassandra/DynamoDB, nem dependências de Bloom Filter (Guava/bloom-filter). A colisão de short codes é resolvida pela unique constraint do PostgreSQL combinada ao retry (RNF01), tornando o Bloom Filter dispensável neste volume. Não incluí Spring Cloud Config, Eureka, Gateway ou Resilience4j completo: para um serviço único read-heavy, o 12-factor via application.yml/env (RNF09) e o spring-retry cobrem os RNFs sem introduzir infraestrutura distribuída injustificada.